

ML-Based Python Code Summarizer

Bareerah Muslim
Matricola: 7175312

June 04, 2026

Contents

1	Introduction	3
1.1	Project Objectives	3
2	Methodology	3
2.1	Dataset	3
2.1.1	Dataset Source	3
2.1.2	Dataset Usage and Train-Validation-Test Split	3
2.2	Data Preprocessing	4
2.2.1	Code Cleaning	4
2.2.2	Summary Cleaning	4
2.2.3	Tokenization	4
2.2.4	Vocabulary Construction	4
2.3	Model Architecture	5
2.4	Training Procedure	5
2.4.1	Loss Function	5
2.4.2	Optimizer	5
2.4.3	Hyperparameters	5
2.4.4	Training Workflow	6
3	Evaluation Metrics	6
4	Results and Discussion	6
4.1	Training Results	6
4.2	Evaluation Results	6
4.3	Generated Summary Examples	7
4.4	Analysis of Results	7
4.5	Limitations	8
4.6	Future Improvements	8
5	Conclusion	8
A	Project Structure	9
B	Group Member Contributions	9

1 Introduction

1.1 Project Objectives

The objective of this project is to build a PyTorch-based tool for generating short natural language summaries of Python functions using code-summary pairs from CodeSearchNet [1, 2]. The input code is cleaned by removing embedded docstrings to avoid data leakage, while the function name is kept as useful context. The target summary is taken from the main descriptive part of the original docstring. A Byte-Level BPE tokenizer is trained on the prepared data, and the summarization model is implemented as an LSTM encoder-decoder with attention. The model is trained from scratch, without using a pretrained full language model, so the project covers the full workflow from preprocessing and tokenization to training, evaluation, and command-line inference.

The main goals of the project are:

- Prepare Python code-summary pairs from CodeSearchNet.
- Remove docstring leakage from the input code.
- Include function names as additional input context.
- Train a BPE tokenizer for code and summary text.
- Implement an LSTM sequence-to-sequence model with attention in PyTorch.
- Train and validate the model using cross-entropy loss and perplexity.
- Evaluate generated summaries using BLEU, ROUGE-1, and ROUGE-L.
- Provide command-line inference for summarizing new Python code snippets.

2 Methodology

2.1 Dataset

2.1.1 Dataset Source

The dataset used in this project is the Python portion of CodeSearchNet, downloaded from the Kaggle CodeSearchNet dataset page [2].

It contains Python functions and their associated docstrings. Each row includes several fields, such as repository name, file path, function name, source code, tokenized code, docstring, tokenized docstring, URL, and partition.

For this project, the most important columns were `code`, `docstring`, and `func_name`. The `code` column was used as the main input, the `docstring` column was used to build the target summary, and `func_name` was added to the input because function names often contain useful information about the purpose of the code.

2.1.2 Dataset Usage and Train-Validation-Test Split

The final training run used **15,000** examples from the CodeSearchNet training split. These examples were divided into training and validation subsets using an **80/20** random split, with a fixed random seed for reproducibility. This gave approximately **12,000** examples for training and **3,000** examples for validation. Final evaluation was performed separately on **1,000** examples from the official CodeSearchNet test split. Since the model was trained on CPU, a limited subset of the full dataset was used to keep training and evaluation time manageable.

2.2 Data Preprocessing

2.2.1 Code Cleaning

Rows with missing or empty code and docstring values were removed. A key preprocessing step was removing docstrings from the input code. This was important because CodeSearchNet functions often contain the docstring inside the function body. If this text remains in the input, the model can indirectly see the reference summary during training, which creates data leakage.

After removing docstrings, the function name was added to the input. For example, the model input was formatted as:

```
function name: is_even
code:
def is_even(n):
    return n % 2 == 0
```

This keeps the input realistic while still giving the model useful information from the function name.

2.2.2 Summary Cleaning

Many docstrings contain long parameter descriptions, return descriptions, and examples. Since the goal of the project was to generate short summaries, the target summary was simplified by keeping the main descriptive sentence of the docstring. This made the target closer to a concise explanation instead of full API documentation.

2.2.3 Tokenization

The final system uses a Byte-Level Byte-Pair Encoding tokenizer implemented with the Hugging Face Tokenizers library [3]. BPE was chosen because it can split rare identifiers and words into smaller subword units. This is useful for code, where names such as `calculate_total` or `model_save_path` may not appear frequently as complete tokens.

The tokenizer was trained on both the model inputs and target summaries. The final vocabulary size was set to 8,000 tokens. Special tokens are added for sequence modeling:

- <PAD> for padding shorter sequences;
- <UNK> for unknown tokens;
- <SOS> for the start of a target sequence;
- <EOS> for the end of a target sequence.

2.2.4 Vocabulary Construction

The BPE tokenizer builds a shared vocabulary for both code and summaries. A shared vocabulary is useful because Python identifiers and summary words often overlap. A smaller vocabulary of 8,000 tokens was used in the final run to reduce the output space and make generation easier for the decoder.

```
BPE_VOCAB_SIZE = 8000
code_vocab = bpe_vocab(tokenizer)
doc_vocab = code_vocab
```

2.3 Model Architecture

The model is a sequence-to-sequence architecture with an LSTM encoder, an LSTM decoder, and an attention mechanism, following the general encoder-decoder idea used in sequence-to-sequence learning [4].

1. Embedding Layer:

The embedding layer converts token IDs into dense vectors. These embeddings are learned during training. The embedding dimension used in the final model was 64.

2. Encoder:

The encoder is an LSTM that reads the input sequence and produces hidden representations for each input token. These encoder outputs are later used by the attention mechanism, so the decoder can refer back to different parts of the input instead of relying only on the final hidden state.

3. Decoder:

The decoder is also an LSTM. It generates the summary one token at a time, starting from the <SOS> token and stopping when it predicts <EOS> or reaches the maximum output length.

4. Attention Mechanism:

The attention mechanism helps the decoder focus on different parts of the encoded input while generating each summary token. At each decoding step, the decoder hidden state is compared with all encoder outputs. The resulting scores are normalized with softmax to produce attention weights. These weights are used to build a context vector, which summarizes the most relevant parts of the input for the current prediction.

5. Output Layer:

The output layer predicts the next summary token. It uses a combination of the decoder LSTM output, the attention context vector, and the previous token embedding, then maps this representation to the output vocabulary size.

2.4 Training Procedure

2.4.1 Loss Function

The model was trained using cross-entropy loss. Padding tokens were ignored in the loss calculation so that padded positions did not affect training.

2.4.2 Optimizer

The Adam optimizer was used with a learning rate of 0.0005. Adam was chosen because it is commonly used for neural sequence models and adapts the learning rate during training.

2.4.3 Hyperparameters

The final configuration used 15,000 training examples, 10 epochs, batch size 32, embedding dimension 64, hidden dimension 128, BPE vocabulary size 8,000, maximum input length 160 tokens, and maximum summary length 25 tokens. Gradient clipping with a maximum norm of 1.0 was applied to reduce the risk of unstable LSTM updates.

2.4.4 Training Workflow

The training script loads the dataset, cleans the data, removes docstrings from the code input, builds the model input using the function name and code, trains the BPE tokenizer, converts text into token IDs, pads the sequences, and trains the model. After each epoch, validation loss and perplexity are calculated. The best model is saved based on validation loss, and the training history is saved for later analysis.

3 Evaluation Metrics

1. **Cross-Entropy Loss:** Cross-entropy loss measures how well the predicted token distribution matches the target summary. Lower loss means the model assigns higher probability to the correct target tokens.
2. **BLEU Score:** BLEU measures n-gram overlap between generated and reference summaries [6]. It is strict, because a correct summary can be phrased differently from the reference.
3. **ROUGE-1 Score:** ROUGE-1 measures word overlap between the generated summary and reference summary [7]. It gives a basic indication of whether important words are shared..
4. **ROUGE-L Score:** ROUGE-L measures the longest common subsequence between generated and reference text [7]. It considers word order more than ROUGE-1.

4 Results and Discussion

4.1 Training Results

In the final run, the training loss decreased from 6.6784 to 5.1822 over 10 epochs. The best validation loss was reached at epoch 8, with a validation loss of 6.2692 and validation perplexity of 528.06. After this point, validation loss increased slightly, so the epoch 8 model was kept as the best model.

This behavior shows that the model learned patterns from the training data, but generalization remained difficult. The decreasing training loss together with relatively high validation loss suggests that the model has limited ability to fully capture code semantics from the available training setup.

4.2 Evaluation Results

The final evaluation used 1,000 examples from the official CodeSearchNet test split. This split was kept separate from training, so the reported metrics reflect the model’s performance on unseen code-summary pairs.

Metric	Value
Average loss	6.4893
Perplexity	658.04
Token accuracy	0.0768
BLEU	2.90×10^{-81}
ROUGE-1	0.1114
ROUGE-L	0.1077
Exact match	0.0

The scores show that the model learned some common summary patterns, but it still has difficulty producing complete and semantically accurate summaries for unseen code. BLEU is especially low because the generated summaries often use different, incomplete, or generic wording compared with the reference summaries.

4.3 Generated Summary Examples

Some manual inference examples are shown below:

Input Function	Generated Summary
<code>def is_even(n): return n % 2 == 0</code>	Check if the
<code>def add_numbers(a, b): return a + b</code>	Return a fully-qualified
<code>def print_log(text): sys.stderr.write(text)</code>	Replace the.
<code>def calculate_total(items): return sum(item.price for item in items)</code>	Return a fully-qualified

The `is_even` example shows that the model sometimes captures part of the function intent. However, other examples are generic. This confirms that the model is a baseline rather than a production-quality summarizer.

These examples also show the qualitative progress of the project. Earlier versions of the model often produced repeated or very generic outputs such as `return the <UNK>`. After adding docstring removal, function-name context, BPE tokenization, attention, and beam search, the outputs became more readable, although they are still often incomplete or inaccurate. For example, the model changed from producing mostly generic phrases to partially meaningful outputs such as `Check if the` for the `is_even` function.

4.4 Analysis of Results

The experiments showed that preprocessing has a major effect on code summarization. When docstrings were left inside the input code, evaluation scores became much higher, but this was misleading because the model could see the target summary. Removing docstrings made the task harder, but it also made the evaluation more realistic.

Several intermediate versions were tested during development. With the earlier word-level tokenizer, the model often generated very generic outputs such as `return the <UNK>` or `return a the`. This showed that the vocabulary and preprocessing were not enough for handling rare code identifiers and summary words.

After switching to BPE tokenization, the outputs became more readable because rare words and identifiers could be split into smaller subword units. Adding function names also gave the model useful semantic information. However, the LSTM-based model still struggled to infer precise behavior from code. Many generated summaries followed frequent dataset patterns, such as “Return...”, instead of accurately describing the input function.

The table below compares an earlier development evaluation with the final evaluation on the official test split.

Experiment	Loss	Perplexity	Accuracy	BLEU	R-1	R-L
Earlier setup	5.8988	364.59	0.0894	1.83×10^{-80}	0.1228	0.1190
Official test	6.4893	658.04	0.0768	2.90×10^{-81}	0.1114	0.1077

The final test scores are lower than the earlier results because the official test split contains unseen examples and gives a more realistic estimate of generalization. For this reason, the official test split results are used as the final reported performance.

4.5 Limitations

The main limitations are that the model was **trained from scratch**, only a **subset of the dataset** was used, and training was limited by CPU resources. The LSTM architecture also has limited capacity compared with modern Transformer models. Another limitation is that automatic metrics such as BLEU and ROUGE do not always reflect whether a summary is semantically correct.

4.6 Future Improvements

Future work could use a Transformer architecture, train on a larger and cleaner dataset, add a copy mechanism for identifiers, and improve decoding. Another useful improvement would be filtering the dataset more carefully so that summaries are shorter and more consistent. If allowed as an extension, a pretrained code model could also be fine-tuned and compared with the custom baseline.

5 Conclusion

This project implemented a PyTorch-based Python code summarization system using an LSTM sequence-to-sequence model with attention. The final pipeline includes CodeSearchNet preprocessing, docstring leakage removal, BPE tokenization, function-name input, training and validation routines, evaluation metrics, and command-line inference. The final model learned some general documentation patterns, but its generated summaries were often generic or incomplete. This shows the difficulty of training a code summarization model from scratch with limited computational resources.

Future work should focus on stronger architectures, especially Transformers, better data filtering, larger training data, and more robust semantic evaluation. Overall, the project provides a complete baseline implementation and highlights the practical challenges of machine-learning-based code summarization.

References

- [1] PyTorch Documentation. <https://pytorch.org/docs>
- [2] CodeSearchNet Dataset on Kaggle. <https://www.kaggle.com/datasets/omduggineni/codesearchnet>
- [3] Hugging Face Tokenizers Documentation. <https://huggingface.co/docs/tokenizers>
- [4] Sutskever, I., Vinyals, O., and Le, Q. V. Sequence to Sequence Learning with Neural Networks. 2014.
- [5] Bahdanau, D., Cho, K., and Bengio, Y. Neural Machine Translation by Jointly Learning to Align and Translate. 2015.
- [6] Papineni, K., Roukos, S., Ward, T., and Zhu, W. J. BLEU: a Method for Automatic Evaluation of Machine Translation. 2002.
- [7] Lin, C.-Y. ROUGE: A Package for Automatic Evaluation of Summaries. 2004.

A Project Structure

```
mlsaProject/  
|  
|-- data/  
|-- models/  
|-- outputs/  
|-- scripts/  
|-- src/  
|-- archive/  
|-- requirements.txt  
|-- README.md  
|-- Report
```

B Group Member Contributions

Member	Contribution
Bareerah Muslim	Project implementation, training, evaluation, and report writing

Table 1: Project Contributions